

# Eksperimen 6 — Analisis Hasil

Notebook ini memuat dan memvisualisasikan hasil training & evaluasi Eksperimen 6.

|                         |                                                      |
|-------------------------|------------------------------------------------------|
| <b>Model</b>            | EfficientNetV2-S (Binary Classification)             |
| <b>View</b>             | Front + Side                                         |
| <b>Fusion</b>           | Average & Adaptive (confidence-based)                |
| <b>Versi Eksperimen</b> | Eksperimen 6 (Ablasi Regularisasi dari Eksperimen 5) |

In [1]:

```
# — Setup —
```

---

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.gridspec as gridspec
```

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Tambahkan root project ke path agar bisa import src.*
```

```
PROJECT_ROOT = Path("../").resolve()
```

```
if str(PROJECT_ROOT) not in sys.path:
```

```
    sys.path.insert(0, str(PROJECT_ROOT))
```

```
RESULTS_DIR = PROJECT_ROOT / "results"
```

```
# Style
```

```
plt.style.use("seaborn-v0_8-darkgrid")
```

```
COLORS = {"front": "#4C9BE8", "side": "#E8834C", "avg": "#6FCB9F", }
```

```
print(" ✓ Setup selesai. Project root:", PROJECT_ROOT)
```

✓ Setup selesai. Project root: D:\Skripsi\Experiment

## 0. Konfigurasi Eksperimen (Eksperimen 5 → Eksperimen 6)

Perubahan parameter dari baseline Eksperimen 5 ke Eksperimen 6 (ablasi regularisasi untuk menangani overfitting):

In [2]:

```
# — Tampilkan diff konfigurasi Eksperimen 5 → Eksperimen 6 —————
```

```
config_diff = [  
    {"Step": "1", "Parameter": "NUM_STAGES_TO_FREEZE", "Exp 5 (Baseline)":  
    {"Step": "2", "Parameter": "WEIGHT_DECAY", "Exp 5 (Baseline)":  
    {"Step": "3", "Parameter": "DROPOUT", "Exp 5 (Baseline)": "0  
    {"Step": "4", "Parameter": "RandomRotation", "Exp 5 (Baseline)": "  
    {"Step": "4", "Parameter": "ColorJitter", "Exp 5 (Baseline)": "0.3"  
]
```

```
df_config = pd.DataFrame(config_diff)  
display(df_config.style  
    .set_properties(**{"text-align": "left"})  
    .set_caption("Perubahan Konfigurasi Eksperimen 5 → Eksperimen 6 (  
    .highlight_between(subset=["Step"], color="#f0f4ff")  
    .hide(axis="index"))
```

| Step | Parameter            | Exp 5<br>(Baseline) | Exp 6<br>(Ablasi) | Alasan                                                        |
|------|----------------------|---------------------|-------------------|---------------------------------------------------------------|
| 1    | NUM_STAGES_TO_FREEZE | 4                   | 5                 | Kurangi parameter trainable; hanya blok terakhir + head aktif |

|   |                |      |      |                                                                 |
|---|----------------|------|------|-----------------------------------------------------------------|
| 2 | WEIGHT_DECAY   | 5e-4 | 1e-3 | Perkuat L2<br>penalty<br>untuk<br>menekan<br>magnitude<br>logit |
| 3 | DROPOUT        | 0.5  | 0.6  | Tambah<br>regularisasi<br>stokastik di<br>head                  |
| 4 | RandomRotation | 15°  | 20°  | Augmentasi<br>lebih<br>agresif<br>untuk<br>generalisasi         |
| 4 | ColorJitter    | 0.3  | 0.4  | Variasi<br>warna<br>lebih besar                                 |

Perubahan Konfigurasi Eksperimen 5 → Eksperimen 6 (Ablasi Regularisasi)

## 0.5. Otomatisasi Training & Evaluasi (Opsional)

Ubah `RUN_RETRAINING` di bawah ini menjadi `True` jika kamu ingin notebook ini otomatis melatih ulang model dan memperbarui data hasil secara langsung saat dijalankan.

In [3]:

```
# ⚠️ PENTING: Set ke True hanya jika ingin melakukan re-training (memerbarui data hasil secara langsung saat dijalankan)
RUN_RETRAINING = True
```

```
if RUN_RETRAINING:
```

```
    print("🦋 Memulai Retraining Model (Konfigurasi Eksperimen 6)...\n")
```

```
    print("1/3 [FRONT VIEW] Melatih model...")
```

```
    # Pengalihan '2>&1' digabung dengan '-u' memastikan semua log stream
```

```
!cd .. && python -u -m src.train --view front 2>&1
```

```
print("\n2/3 [SIDE VIEW] Melatih model...")
```

```
!cd .. && python -u -m src.train --view side 2>&1
```

```
print("\n3/3 [FUSION & EVALUATION] Menjalankan fusion...")
```

```
!cd .. && python -u -m src.fusion 2>&1
```

```
print("\n✓ Re-training selesai! Seluruh hasil di folder results/ telah  
else:
```

```
print("i Re-training dilewati (RUN_RETRAINING = False).")
```

```
print(" Notebook akan memvisualisasikan data yang sudah ada di fo
```

🐦 Memulai Retraining Model (Konfigurasi Eksperimen 6)...

1/3 [FRONT VIEW] Melatih model...

INFO:

=====

INFO: Experiment Configuration

INFO: =====

INFO: View : front

INFO: Optimizer : AdamW

INFO: Learning Rate : 5.0e-06

INFO: Weight Decay : 1.0e-03

INFO: Scheduler : ReduceLROnPlateau

INFO: Dropout : 0.6

INFO: Frozen Stages : 5

INFO: EarlyStopping : patience=5

INFO: Experiment ID : None

INFO: =====

INFO: Device: cuda

INFO: GPU: NVIDIA GeForce RTX 5060 Laptop GPU

INFO: Frame subsampling diterapkan (stride=5): 5976 frame tersisa unt

INFO: DataLoader dibuat: view=front split=train n=5976 batch\_size=

INFO: Frame subsampling diterapkan (stride=5): 1249 frame tersisa unt

INFO: DataLoader dibuat: view=front split=val n=1249 batch\_size=32

INFO: Frame subsampling diterapkan (stride=5): 1215 frame tersisa unt

INFO: DataLoader dibuat: view=front split=test n=1215 batch\_size=32

INFO: Loading pretrained weights from Hugging Face hub (timm/tf\_effic

INFO: HTTP Request: HEAD https://huggingface.co/timm/tf\_efficientnet  
Warning: You are sending unauthenticated requests to the HF Hub. Please  
WARNING: Warning: You are sending unauthenticated requests to the HF Hub.  
INFO: [timm/tf\_efficientnetv2\_s.in21k\_ft\_in1k] Safe alternative available  
INFO: Membekukan conv\_stem, bn1, dan 5 stage block pertama pada backbone  
INFO: =====  
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)  
INFO: =====  
INFO: backbone.conv\_stem : FROZEN  
INFO: backbone.bn1 : FROZEN  
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)  
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)  
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)  
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)  
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)  
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)  
INFO: backbone.conv\_head : TRAINABLE  
INFO: backbone.bn2 : TRAINABLE  
INFO: classifier : TRAINABLE  
INFO: =====  
INFO: Model: tf\_efficientnetv2\_s | Total params: 20,180,050 | Trainable: 15,100,000  
INFO: Frame subsampling diterapkan (stride=5): 5976 frame tersisa untuk training  
INFO: Distribusi kelas training (front): safe\_driving=963, phone\_use=5000  
INFO: Class weights (CrossEntropyLoss): safe\_driving=3.1028, phone\_use=0.0001  
INFO: =====  
INFO: Training: view = front | max\_epochs = 30 | patience = 5  
INFO: =====  
INFO: Epoch 01/30 | train\_loss=0.7455 | val\_loss=0.6259 | val\_F1=0.59  
INFO: Epoch 02/30 | train\_loss=0.6518 | val\_loss=0.5888 | val\_F1=0.60  
INFO: Epoch 03/30 | train\_loss=0.5938 | val\_loss=0.5543 | val\_F1=0.61  
INFO: Epoch 04/30 | train\_loss=0.5261 | val\_loss=0.5403 | val\_F1=0.62  
INFO: Epoch 05/30 | train\_loss=0.4869 | val\_loss=0.5286 | val\_F1=0.63  
INFO: Epoch 06/30 | train\_loss=0.4396 | val\_loss=0.4802 | val\_F1=0.70  
INFO: Epoch 07/30 | train\_loss=0.3964 | val\_loss=0.4791 | val\_F1=0.71  
INFO: Epoch 08/30 | train\_loss=0.3759 | val\_loss=0.4502 | val\_F1=0.72  
INFO: Epoch 09/30 | train\_loss=0.3395 | val\_loss=0.4460 | val\_F1=0.73  
INFO: Epoch 10/30 | train\_loss=0.3283 | val\_loss=0.4072 | val\_F1=0.74  
INFO: Epoch 11/30 | train\_loss=0.2959 | val\_loss=0.3992 | val\_F1=0.75  
INFO: Epoch 12/30 | train\_loss=0.2782 | val\_loss=0.3935 | val\_F1=0.76  
INFO: Epoch 13/30 | train\_loss=0.2539 | val\_loss=0.3850 | val\_F1=0.77

|                   |                     |                   |              |
|-------------------|---------------------|-------------------|--------------|
| INFO: Epoch 14/30 | train_loss = 0.2423 | val_loss = 0.3824 | val_F1 = 0.7 |
| INFO: Epoch 15/30 | train_loss = 0.2353 | val_loss = 0.3843 | val_F1 = 0.7 |
| INFO: Epoch 16/30 | train_loss = 0.2272 | val_loss = 0.3676 | val_F1 = 0.7 |
| INFO: Epoch 17/30 | train_loss = 0.2277 | val_loss = 0.3706 | val_F1 = 0.7 |
| INFO: Epoch 18/30 | train_loss = 0.2028 | val_loss = 0.3708 | val_F1 = 0.7 |
| INFO: Epoch 19/30 | train_loss = 0.2059 | val_loss = 0.3569 | val_F1 = 0.7 |
| INFO: Epoch 20/30 | train_loss = 0.2008 | val_loss = 0.3553 | val_F1 = 0.7 |
| INFO: Epoch 21/30 | train_loss = 0.1936 | val_loss = 0.3571 | val_F1 = 0.7 |
| INFO: Epoch 22/30 | train_loss = 0.2053 | val_loss = 0.3539 | val_F1 = 0.7 |
| INFO: Epoch 23/30 | train_loss = 0.1788 | val_loss = 0.3502 | val_F1 = 0.7 |
| INFO: Epoch 24/30 | train_loss = 0.1775 | val_loss = 0.3459 | val_F1 = 0.7 |
| INFO: Epoch 25/30 | train_loss = 0.1758 | val_loss = 0.3592 | val_F1 = 0.7 |
| INFO: Epoch 26/30 | train_loss = 0.1636 | val_loss = 0.3741 | val_F1 = 0.7 |
| INFO: Epoch 27/30 | train_loss = 0.1575 | val_loss = 0.3556 | val_F1 = 0.7 |
| INFO: Epoch 28/30 | train_loss = 0.1496 | val_loss = 0.3538 | val_F1 = 0.7 |

INFO: Early stopping triggered at epoch 28. Best epoch: 23 (val\_F1 = 0.793)

INFO: Training selesai dalam 23.8 menit. Best epoch: 23 (val\_F1 = 0.793)

INFO: Checkpoint tersimpan di: D:\Skripsi\Experiment\checkpoints\front\_checkpoint\_23.pt

INFO: History tersimpan di: D:\Skripsi\Experiment\results\front\_history.pkl

2/3 [SIDE VIEW] Melatih model...

INFO:

=====

INFO: Experiment Configuration

INFO: =====

INFO: View : side

INFO: Optimizer : AdamW

INFO: Learning Rate : 5.0e-06

INFO: Weight Decay : 1.0e-03

INFO: Scheduler : ReduceLROnPlateau

INFO: Dropout : 0.6

INFO: Frozen Stages : 5

INFO: EarlyStopping : patience=5

INFO: Experiment ID : None

INFO: =====

INFO: Device: cuda

INFO: GPU: NVIDIA GeForce RTX 5060 Laptop GPU

INFO: Frame subsampling diterapkan (stride=5): 5976 frame tersisa untuk pelatihan

```

INFO: DataLoader dibuat: view = side split = train n = 5976 batch_size = 32
INFO: Frame subsampling diterapkan (stride = 5): 1249 frame tersisa untuk training
INFO: DataLoader dibuat: view = side split = val n = 1249 batch_size = 32
INFO: Frame subsampling diterapkan (stride = 5): 1215 frame tersisa untuk validasi
INFO: DataLoader dibuat: view = side split = test n = 1215 batch_size = 32
INFO: Loading pretrained weights from Hugging Face hub (timm/tf_efficientnetv2_s)
INFO: HTTP Request: GET https://huggingface.co/api/agent-harnesses "https://huggingface.co/api/agent-harnesses"
INFO: HTTP Request: HEAD https://huggingface.co/timm/tf_efficientnetv2_s.in21k_ft_in1k
INFO: [timm/tf_efficientnetv2_s.in21k_ft_in1k] Safe alternative available
INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1 : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2 : TRAINABLE
INFO: classifier : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable: 10,180,050
INFO: Frame subsampling diterapkan (stride = 5): 5976 frame tersisa untuk training
INFO: Distribusi kelas training (side): safe_driving = 963, phone_use = 50, other = 1504
INFO: Class weights (CrossEntropyLoss): safe_driving = 3.1028, phone_use = 1.5972, other = 1.0
INFO: =====
INFO: Training: view = side | max_epochs = 30 | patience = 5
INFO: =====
INFO: Epoch 01/30 | train_loss = 0.8139 | val_loss = 0.6651 | val_F1 = 0.4651
INFO: Epoch 02/30 | train_loss = 0.7134 | val_loss = 0.6223 | val_F1 = 0.5000
INFO: Epoch 03/30 | train_loss = 0.6901 | val_loss = 0.6083 | val_F1 = 0.5000
INFO: Epoch 04/30 | train_loss = 0.6505 | val_loss = 0.5962 | val_F1 = 0.5000
INFO: Epoch 05/30 | train_loss = 0.5924 | val_loss = 0.5874 | val_F1 = 0.5000
INFO: Epoch 06/30 | train_loss = 0.5590 | val_loss = 0.5434 | val_F1 = 0.6000
INFO: Epoch 07/30 | train_loss = 0.5000 | val_loss = 0.5532 | val_F1 = 0.5000
INFO: Epoch 08/30 | train_loss = 0.4642 | val_loss = 0.5265 | val_F1 = 0.6000

```

```

INFO: Epoch 09/30 | train_loss = 0.4282 | val_loss = 0.4924 | val_F1 = 0.6
INFO: Epoch 10/30 | train_loss = 0.3950 | val_loss = 0.4768 | val_F1 = 0.6
INFO: Epoch 11/30 | train_loss = 0.3444 | val_loss = 0.4768 | val_F1 = 0.6
INFO: Epoch 12/30 | train_loss = 0.3200 | val_loss = 0.4717 | val_F1 = 0.6
INFO: Epoch 13/30 | train_loss = 0.2820 | val_loss = 0.4495 | val_F1 = 0.6
INFO: Epoch 14/30 | train_loss = 0.2546 | val_loss = 0.4561 | val_F1 = 0.6
INFO: Epoch 15/30 | train_loss = 0.2295 | val_loss = 0.4329 | val_F1 = 0.6
INFO: Epoch 16/30 | train_loss = 0.2163 | val_loss = 0.4365 | val_F1 = 0.7
INFO: Epoch 17/30 | train_loss = 0.2006 | val_loss = 0.4310 | val_F1 = 0.7
INFO: Epoch 18/30 | train_loss = 0.1783 | val_loss = 0.4343 | val_F1 = 0.7
INFO: Epoch 19/30 | train_loss = 0.1593 | val_loss = 0.4281 | val_F1 = 0.7
INFO: Epoch 20/30 | train_loss = 0.1504 | val_loss = 0.4425 | val_F1 = 0.6
INFO: Epoch 21/30 | train_loss = 0.1403 | val_loss = 0.4426 | val_F1 = 0.7
INFO: Epoch 22/30 | train_loss = 0.1387 | val_loss = 0.4657 | val_F1 = 0.7
INFO: Epoch 23/30 | train_loss = 0.1296 | val_loss = 0.4721 | val_F1 = 0.6
INFO: Epoch 24/30 | train_loss = 0.1287 | val_loss = 0.4605 | val_F1 = 0.7
INFO: Early stopping triggered at epoch 24. Best epoch: 19 (val_F1 = 0.728)
INFO: =====
INFO: Training selesai dalam 20.0 menit. Best epoch: 19 (val_F1 = 0.728)
INFO: Checkpoint tersimpan di: D:\Skripsi\Experiment\checkpoints\side_history.j
INFO: History tersimpan di: D:\Skripsi\Experiment\results\side_history.j

```

3/3 [FUSION & EVALUATION] Menjalankan fusion...

Class names: ['safe\_driving', 'phone\_use']

INFO: Device: cuda

INFO: Membekukan conv\_stem, bn1, dan 5 stage block pertama pada backbone

INFO: =====

INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)

INFO: =====

INFO: backbone.conv\_stem : FROZEN

INFO: backbone.bn1 : FROZEN

INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)

INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)

INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)

INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)

INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)

INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)

INFO: backbone.conv\_head : TRAINABLE

INFO: backbone.bn2 : TRAINABLE

```
INFO: classifier      : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Loaded front checkpoint (default): epoch=23, val_F1=0.7938
INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier      : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Loaded side checkpoint (default): epoch=19, val_F1=0.7283
INFO: Frame subsampling diterapkan pada paired test loader (stride=5)
INFO: Paired test DataLoader: n=1215 pairs, batch_size=32
INFO: Running inference on paired test set...
Class names: ['safe_driving', 'phone_use']
Class names: ['safe_driving', 'phone_use']
Class names: ['safe_driving', 'phone_use']
Class names: ['safe_driving', 'phone_use']
INFO: Inference selesai: 1215 paired frames
INFO:
INFO: =====
INFO: TEST RESULTS: SINGLE-VIEW FRONT
INFO: =====
INFO: Accuracy : 0.8971
INFO: Precision: 0.8254 (macro)
INFO: Recall   : 0.8333 (macro)
INFO: F1-Score : 0.8293 (macro)
INFO: Confusion Matrix:
```

INFO:           Pred:safe Pred:phone  
INFO: True:safe    162     59  
INFO: True:phone   66     928

INFO: -----

INFO: = = = = =

INFO: TEST RESULTS: SINGLE-VIEW SIDE

INFO: = = = = =

INFO: Accuracy : 0.8420

INFO: Precision: 0.7343 (macro)

INFO: Recall : 0.7310 (macro)

INFO: F1-Score : 0.7326 (macro)

INFO: Confusion Matrix:

INFO:           Pred:safe Pred:phone

INFO: True:safe    123     98

INFO: True:phone   94     900

INFO: -----

INFO: = = = = =

INFO: TEST RESULTS: AVERAGE FUSION (50:50)

INFO: = = = = =

INFO: Accuracy : 0.9062

INFO: Precision: 0.8631 (macro)

INFO: Recall : 0.8037 (macro)

INFO: F1-Score : 0.8287 (macro)

INFO: Confusion Matrix:

INFO:           Pred:safe Pred:phone

INFO: True:safe    142     79

INFO: True:phone   35     959

INFO: -----

INFO: = = = = =

INFO: TEST RESULTS: ADAPTIVE FUSION (CONFIDENCE)

INFO: = = = = =

INFO: Accuracy : 0.9062

INFO: Precision: 0.8631 (macro)

INFO: Recall : 0.8037 (macro)

INFO: F1-Score : 0.8287 (macro)

INFO: Confusion Matrix:

INFO:           Pred:safe Pred:phone

INFO: True:safe    142     79

INFO: True:phone   35     959

INFO: -----

INFO:

INFO: = = = = =

INFO: PERBANDINGAN AKHIR (Test Set — Paired Frames)

INFO: = = = = =

| INFO: Method | Acc | Prec | Rec | F1 |
|--------------|-----|------|-----|----|
|--------------|-----|------|-----|----|

INFO: -----

|                         |        |        |        |        |
|-------------------------|--------|--------|--------|--------|
| INFO: Single-view FRONT | 0.8971 | 0.8254 | 0.8333 | 0.8293 |
|-------------------------|--------|--------|--------|--------|

|                        |        |        |        |        |
|------------------------|--------|--------|--------|--------|
| INFO: Single-view SIDE | 0.8420 | 0.7343 | 0.7310 | 0.7326 |
|------------------------|--------|--------|--------|--------|

|                              |        |        |        |       |
|------------------------------|--------|--------|--------|-------|
| INFO: Average Fusion (50:50) | 0.9062 | 0.8631 | 0.8037 | 0.828 |
|------------------------------|--------|--------|--------|-------|

|                                    |        |        |        |      |
|------------------------------------|--------|--------|--------|------|
| INFO: Adaptive Fusion (confidence) | 0.9062 | 0.8631 | 0.8037 | 0.82 |
|------------------------------------|--------|--------|--------|------|

INFO: = = = = =

INFO: Hasil disimpan di: D:\Skripsi\Experiment\results\fusion\_comparis

✓ Re-training selesai! Seluruh hasil di folder results/ telah terupdate.

## 1. Load Hasil Eksperimen

In [4]:

```
# — Load semua file JSON
```

```
with open(RESULTS_DIR / "front_history.json") as f:
```

```
    hist_front = json.load(f)
```

```
with open(RESULTS_DIR / "side_history.json") as f:
```

```
    hist_side = json.load(f)
```

```
with open(RESULTS_DIR / "front_test_metrics.json") as f:
```

```
    metrics_front = json.load(f)
```

```
with open(RESULTS_DIR / "side_test_metrics.json") as f:
```

```
    metrics_side = json.load(f)
```

```
with open(RESULTS_DIR / "fusion_comparison.json") as f:
```

```
    fusion = json.load(f)
```

```
print(f'Front training: {len(hist_front["train_loss"])} epochs')
```

```
print(f'Side training: {len(hist_side["train_loss"])} epochs')
```

```
print(f'\nFront best epoch : {metrics_front["best_epoch"]}')'
```

```
print(f'Front val F1    : {metrics_front['val_macro_f1']:.4f}')
print(f'Side best epoch : {metrics_side['best_epoch']}')
print(f'Side val F1    : {metrics_side['val_macro_f1']:.4f}')
```

Front training: 28 epochs

Side training: 24 epochs

Front best epoch : 24

Front val F1 : 0.8018

Side best epoch : 23

Side val F1 : 0.8140

## 2. Kurva Training

In [5]:

```
# — Kurva Training: Loss & F1 —————
fig, axes = plt.subplots(2, 2, figsize = (14, 9))
fig.suptitle("Kurva Training — Eksperimen 6", fontsize = 15, fontweight = "bold")

for col, (hist, view) in enumerate([(hist_front, "front"), (hist_side, "side")]):
    epochs = range(1, len(hist["train_loss"]) + 1)
    c = COLORS[view]

    # Loss
    ax = axes[0, col]
    ax.plot(epochs, hist["train_loss"], label = "Train Loss", color = c, linewidth = 2)
    ax.plot(epochs, hist["val_loss"], label = "Val Loss", color = c, linewidth = 2)
    ax.set_title(f'Loss — {view.capitalize()} View', fontweight = "bold")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.legend()

    # F1
    ax = axes[1, col]
    ax.plot(epochs, hist["val_macro_f1"], label = "Val Macro F1", color = c, linewidth = 2)
    ax.axhline(max(hist["val_macro_f1"]), color = c, linestyle = ":", alpha = 0.5,
                label = f'Best F1 = {max(hist["val_macro_f1"]):.4f}')
    ax.set_title(f'Val Macro F1 — {view.capitalize()} View', fontweight = "bold")
```

```
ax.set_xlabel("Epoch")
ax.set_ylabel("Macro F1")
ax.set_ylim(0.4, 1.0)
ax.legend()
```

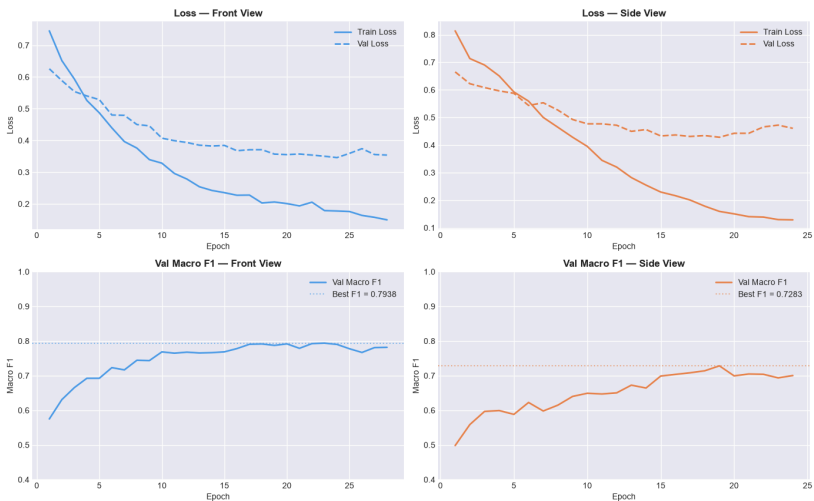
```
plt.tight_layout()
```

```
plt.savefig(RESULTS_DIR / "exp6_training_curves.png", dpi = 150, bbox_
```

```
plt.show()
```

```
print(" ✓ Disimpan ke results/exp6_training_curves.png")
```

Kurva Training — Eksperimen 6



✓ Disimpan ke results/exp6\_training\_curves.png

## 2b. Analisis Gap Train Loss vs Val Loss

Gap besar ( $\text{val\_loss} - \text{train\_loss} > 0.15$ ) mengindikasikan **overfitting**, bukan legitimate overconfidence.

In [6]:

```
# — Analisis gap train/val loss per epoch —————
```

```
fig, axes = plt.subplots(1, 2, figsize = (14, 5))
```

```
fig.suptitle("Gap Train Loss vs Val Loss — Eksperimen 6", fontsize = 13,
```

```
gap_summary = []
```

```
for ax, (hist, view) in zip(axes, [(hist_front, "front"), (hist_side, "side")]):  
    train_losses = np.array(hist["train_loss"])  
    val_losses   = np.array(hist["val_loss"])  
    gaps        = val_losses - train_losses  
    epochs      = range(1, len(train_losses) + 1)  
    best_ep_idx = int(np.argmax(hist["val_macro_f1"]))  
    c = COLORS[view]
```

```
ax.plot(epochs, train_losses, label="Train Loss", color=c, linewidth=2)  
ax.plot(epochs, val_losses, label="Val Loss", color=c, linewidth=2)  
ax.fill_between(epochs, train_losses, val_losses, alpha=0.15, color=c)  
ax.axvline(best_ep_idx + 1, color="gray", linestyle=":", linewidth=2)
```

```
final_gap = gaps[-1]  
flag = "⚠️ OVERFIT" if final_gap > 0.15 else ("✓ OK" if final_gap < 0.15 else "🔍")  
ax.set_title(f"{view.capitalize()} View — Gap akhir: {final_gap:.4f} {flag}")  
ax.set_xlabel("Epoch")  
ax.set_ylabel("Loss")  
ax.legend(fontsize=9)
```

```
gap_summary.append({  
    "View": view.capitalize(),  
    "Train Loss (akhir)": round(train_losses[-1], 5),  
    "Val Loss (akhir)": round(val_losses[-1], 5),  
    "Gap akhir": round(final_gap, 5),  
    "Gap di best epoch": round(gaps[best_ep_idx], 5),  
    "Gap maksimum": round(gaps.max(), 5),  
    "Status": flag,  
})
```

```
plt.tight_layout()  
plt.show()
```

```
df_gap = pd.DataFrame(gap_summary).set_index("View")  
display(df_gap.style.set_caption("Ringkasan Gap Loss — Eksperimen 6"))
```

```
C:\Users\adann\AppData\Local\Temp\ipykernel_4328\3490263243.py:  
plt.tight_layout()
```

```
d:\Skripsi\Experiment\.venv\Lib\site-packages\IPython\core\pylabtools
fig.canvas.print_figure(bytes_io, **kw)
```



|       | Train Loss (akhir) | Val Loss (akhir) | Gap akhir | Gap di best epoch | Gap maksimum | Status  |
|-------|--------------------|------------------|-----------|-------------------|--------------|---------|
| View  |                    |                  |           |                   |              |         |
| Front | 0.149600           | 0.353840         | 0.204240  | 0.171340          | 0.210510     | OVERFIT |
| Side  | 0.128700           | 0.460510         | 0.331810  | 0.268820          | 0.342530     | OVERFIT |

Ringkasan Gap Loss — Eksperimen 6

### 3. Confusion Matrix — Test Set

```
In [7]:
# — Confusion Matrix: 4 metode sekaligus
CLASS_NAMES = ["safe_driving", "phone_use"]

methods = [
    ("Single-View Front", fusion["single_front"], COLORS["front"]),
    ("Single-View Side", fusion["single_side"], COLORS["side"]),
    ("Average Fusion (50:50)", fusion["average_fusion"], COLORS["avg"]),
    ("Adaptive Fusion (confidence)", fusion["adaptive_fusion"], COLORS["a"])
]
```

```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
fig.suptitle("Confusion Matrix — Test Set (Eksperimen 6)", fontsize=14)
```

```

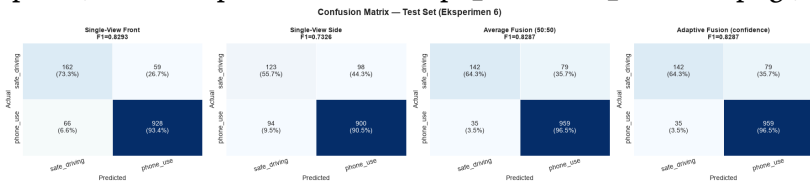
for ax, (title, m, color) in zip(axes, methods):
    cm = np.array(m["confusion_matrix"])
    cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100

    annot = np.array([[f'{v}\n({p:.1f}%)' for v, p in zip(row_v, row_p)]
                      for row_v, row_p in zip(cm, cm_pct)])

    sns.heatmap(cm, annot=annot, fmt="", ax=ax, cmap="Blues",
                xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
                linewidths=0.5, cbar=False)
    ax.set_title(f'{title}\nF1 = {m['f1_macro']:.4f}', fontweight="bold", fontstyle="italic")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.tick_params(axis="x", rotation=15)

plt.tight_layout()
plt.savefig(RESULTS_DIR / "exp6_confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()
print("✓ Disimpan ke results/exp6_confusion_matrices.png")

```



✓ Disimpan ke results/exp6\_confusion\_matrices.png

## 4. Tabel Perbandingan Metrik — Test Set

In [8]:

```

# — Tabel perbandingan semua metode —————
rows = []
for key, label in [
    ("single_front", "Single-View Front"),
    ("single_side", "Single-View Side"),
    ("average_fusion", "Average Fusion (50:50)"),
    ("adaptive_fusion", "Adaptive Fusion (confidence)"),

```

```
]:
m = fusion[key]
rows.append({
    "Method": label,
    "Accuracy": f'{m["accuracy"]:.4f}',
    "Precision (macro)": f'{m["precision_macro"]:.4f}',
    "Recall (macro)": f'{m["recall_macro"]:.4f}',
    "F1 (macro)": f'{m["f1_macro"]:.4f}',
})

df_summary = pd.DataFrame(rows).set_index("Method")
display(df_summary.style
    .highlight_max(axis=0, props="font-weight:bold; color:white; background-color: #f2f2f2")
    .set_caption("Test Set — Eksperimen 6"))
```

|                                             | Accuracy | Precision<br>(macro) | Recall<br>(macro) | F1<br>(macro) |
|---------------------------------------------|----------|----------------------|-------------------|---------------|
| <b>Method</b>                               |          |                      |                   |               |
| <b>Single-View<br/>Front</b>                | 0.8971   | 0.8254               | 0.8333            | 0.8293        |
| <b>Single-View<br/>Side</b>                 | 0.8420   | 0.7343               | 0.7310            | 0.7326        |
| <b>Average Fusion<br/>(50:50)</b>           | 0.9062   | 0.8631               | 0.8037            | 0.8287        |
| <b>Adaptive<br/>Fusion<br/>(confidence)</b> | 0.9062   | 0.8631               | 0.8037            | 0.8287        |

Test Set — Eksperimen 6

## 5. Bar Chart Perbandingan F1

In [9]:

```
# — Bar chart Macro F1 —————
methods_labels = [
    "Front\n(single)",
    "Side\n(single)",
    "Average\nFusion",
    "Adaptive\nFusion",
```

```

]
f1_scores = [
    fusion["single_front"]["f1_macro"],
    fusion["single_side"]["f1_macro"],
    fusion["average_fusion"]["f1_macro"],
    fusion["adaptive_fusion"]["f1_macro"],
]
bar_colors = [COLORS["front"], COLORS["side"], COLORS["avg"], COLORS["adaptive"]]

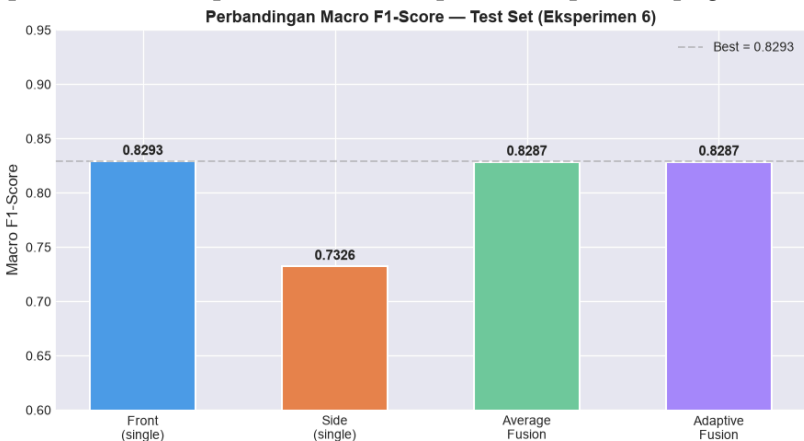
fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(methods_labels, f1_scores, color=bar_colors, width=0.5)

for bar, val in zip(bars, f1_scores):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.003,
            f'{val:.4f}', ha="center", va="bottom", fontweight="bold", fontsize=10)

ax.set_ylim(0.6, 0.95)
ax.set_ylabel("Macro F1-Score", fontsize=12)
ax.set_title("Perbandingan Macro F1-Score — Test Set (Eksperimen 6)",
            fontsize=13, fontweight="bold")
ax.axhline(max(f1_scores), color="gray", linestyle="--", alpha=0.4, label="Best")
ax.legend(fontsize=10)

plt.tight_layout()
plt.savefig(RESULTS_DIR / "exp6_f1_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
print("✓ Disimpan ke results/exp6_f1_comparison.png")

```



✓ Disimpan ke results/exp6\_f1\_comparison.png

## 6. Classification Report Per Kelas

In [10]:

```
# — Classification report detail per kelas
```

```
report_rows = []
```

```
for key, label in [
```

```
    ("single_front", "Front (single)"),
```

```
    ("single_side", "Side (single)"),
```

```
    ("average_fusion", "Average Fusion"),
```

```
    ("adaptive_fusion", "Adaptive Fusion"),
```

```
]:
```

```
    rep = fusion[key]["classification_report"]
```

```
    for cls in ["safe_driving", "phone_use"]:
```

```
        report_rows.append({
```

```
            "Method": label,
```

```
            "Class": cls,
```

```
            "Precision": round(rep[cls]["precision"], 4),
```

```
            "Recall": round(rep[cls]["recall"], 4),
```

```
            "F1-Score": round(rep[cls]["f1-score"], 4),
```

```
            "Support": int(rep[cls]["support"]),
```

```
        })
```

```
df_report = pd.DataFrame(report_rows)
```

```
display(df_report.style
```

```
    .background_gradient(subset = ["Precision", "Recall", "F1-Score"], cmap
```

```
    .set_caption("Classification Report per Kelas — Test Set"))
```

|   | Method         | Class        | Precision | Recall   | F1-Score | Support |
|---|----------------|--------------|-----------|----------|----------|---------|
| 0 | Front (single) | safe_driving | 0.710500  | 0.733000 | 0.721600 | 221     |
| 1 | Front (single) | phone_use    | 0.940200  | 0.933600 | 0.936900 | 994     |
| 2 | Side (single)  | safe_driving | 0.566800  | 0.556600 | 0.561600 | 221     |
| 3 | Side (single)  | phone_use    | 0.901800  | 0.905400 | 0.903600 | 994     |

- 4 Average safe\_driving 0.802300 0.642500 0.713600 221 Fusion
- 5 Average phone\_use 0.923900 0.964800 0.943900 994 Fusion
- 6 Adaptive safe\_driving 0.802300 0.642500 0.713600 221 Fusion
- 7 Adaptive phone\_use 0.923900 0.964800 0.943900 994 Fusion

Classification Report per Kelas — Test Set

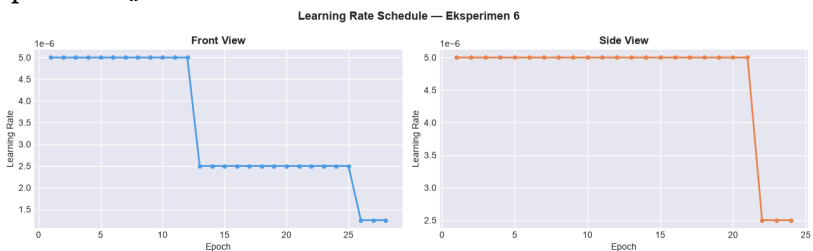
## 7. Learning Rate Schedule

In [11]:

```
# — LR Schedule kedua model —————
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle("Learning Rate Schedule — Eksperimen 6", fontsize=13, fontweight="bold")

for ax, (hist, view) in zip(axes, [(hist_front, "front"), (hist_side, "side")]):
    epochs = range(1, len(hist["lr"]) + 1)
    ax.plot(epochs, hist["lr"], color=COLORS[view], linewidth=2, marker="o")
    ax.set_title(f'{view.capitalize()} View', fontweight="bold")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Learning Rate")
    ax.ticklabel_format(style="sci", axis="y", scilimits=(0, 0))
```

```
plt.tight_layout()
plt.show()
```



## 8. Diagnostik Kalibrasi Probabilitas

## (ECE & Brier Score)

Sel ini menghitung **Expected Calibration Error (ECE)** dan **Brier Score** dari skor prediksi model.

Dibutuhkan checkpoint model — pastikan sudah ada di `checkpoints/front_best.pt` dan `checkpoints/side_best.pt`.

| Metrik                      | Interpretasi                   |
|-----------------------------|--------------------------------|
| ECE $\rightarrow$ 0         | Model well-calibrated          |
| ECE $>$ 0.10                | Overconfident / underconfident |
| Brier Score $\rightarrow$ 0 | Prediksi probabilitas akurat   |
| Brier Score = 0.25          | Baseline prediksi acak         |

In [12]:

```
# — Fungsi ECE & Brier Score (inline, sama persis dengan fusion_debug.py)
def compute_ece(probs: np.ndarray, labels: np.ndarray, n_bins: int = 10)
    """ECE = sum_b (|B_b|/n) * |accuracy(B_b) - confidence(B_b)|"""
    bin_edges = np.linspace(0.0, 1.0, n_bins + 1)
    ece = 0.0
    n = len(probs)
    for i in range(n_bins):
        lo, hi = bin_edges[i], bin_edges[i + 1]
        mask = (probs >= lo) & (probs < hi)
        if mask.sum() == 0:
            continue
        bin_acc = labels[mask].mean()
        bin_conf = probs[mask].mean()
        ece += (mask.sum() / n) * abs(bin_acc - bin_conf)
    return float(ece)

def compute_brier_score(probs: np.ndarray, labels: np.ndarray) -> float
    """BS = (1/n) * sum((p_i - y_i)^2)"""
    return float(np.mean((probs - labels) ** 2))

def reliability_diagram(ax, probs, labels, title, color, n_bins = 10):
```

```
"""Reliability diagram (calibration curve) untuk satu model."""
```

```
bin_edges = np.linspace(0.0, 1.0, n_bins + 1)
```

```
bin_centers, bin_acc, bin_count = [], [], []
```

```
for i in range(n_bins):
```

```
    lo, hi = bin_edges[i], bin_edges[i + 1]
```

```
    mask = (probs >= lo) & (probs < hi)
```

```
    if mask.sum() == 0:
```

```
        continue
```

```
    bin_centers.append(probs[mask].mean())
```

```
    bin_acc.append(labels[mask].mean())
```

```
    bin_count.append(mask.sum())
```

```
bin_centers = np.array(bin_centers)
```

```
bin_acc = np.array(bin_acc)
```

```
ax.plot([0, 1], [0, 1], "k--", alpha=0.5, label="Perfect calibration")
```

```
ax.bar(bin_centers, bin_acc, width=0.08, alpha=0.5, color=color, label=bin_count)
```

```
ax.plot(bin_centers, bin_acc, "o-", color=color, linewidth=2)
```

```
ax.set_xlim(0, 1)
```

```
ax.set_ylim(0, 1)
```

```
ax.set_xlabel("Mean Predicted Confidence")
```

```
ax.set_ylabel("Fraction of Positives (Accuracy)")
```

```
ax.set_title(title, fontweight="bold")
```

```
ax.legend(fontsize=8)
```

```
print("✓ Fungsi kalibrasi siap.")
```

✓ Fungsi kalibrasi siap.

In [13]:

```
# — Jalankan inferensi untuk mendapatkan skor probabilitas —————
```

```
# ⚠️ Sel ini membutuhkan GPU/CPU dan memuat checkpoint model
```

```
import torch
```

```
from src.evaluate import load_trained_model
```

```
from src.fusion import get_paired_test_loader, get_paired_scores
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(f"Device: {device}")
```

```
model_front, _ = load_trained_model("front", device)
```

```

model_side, _ = load_trained_model("side", device)
paired_loader = get_paired_test_loader()

scores_front, scores_side, labels = get_paired_scores(
    model_front, model_side, paired_loader, device
)
labels = labels.astype(float)
print(f"✓ Inferensi selesai: {len(labels)} sampel")

```

```

Class names: ['safe_driving', 'phone_use']
Device: cuda

```

```

INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier         : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable: 15,180,050
INFO: Loaded front checkpoint (default): epoch=23, val_F1=0.7938
INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)

```

```

INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2      : TRAINABLE
INFO: classifier       : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Loaded side checkpoint (default): epoch = 19, val_F1 = 0.7283
INFO: Frame subsampling diterapkan pada paired test loader (stride = 5)
INFO: Paired test DataLoader: n = 1215 pairs, batch_size = 32

```

✓ Inferensi selesai: 1215 sampel

In [14]:

```

# — Hitung & Tampilkan ECE + Brier Score —————
ece_front = compute_ece(scores_front, labels)
ece_side  = compute_ece(scores_side, labels)
bs_front  = compute_brier_score(scores_front, labels)
bs_side   = compute_brier_score(scores_side, labels)

```

*# Tabel ringkasan*

```

df_calib = pd.DataFrame([
    {"View": "Front", "ECE": round(ece_front, 5), "Brier Score": round(bs_front, 5),
      "ECE Status": "✓ BAIK" if ece_front < 0.05 else ("⚠ SEDANG" if ece_front < 0.10 else "❌ BURUK"),
      "BS Status": "✓ BAIK" if bs_front < 0.10 else ("⚠ SEDANG" if bs_front < 0.20 else "❌ BURUK")},
    {"View": "Side", "ECE": round(ece_side, 5), "Brier Score": round(bs_side, 5),
      "ECE Status": "✓ BAIK" if ece_side < 0.05 else ("⚠ SEDANG" if ece_side < 0.10 else "❌ BURUK"),
      "BS Status": "✓ BAIK" if bs_side < 0.10 else ("⚠ SEDANG" if bs_side < 0.20 else "❌ BURUK")}
]).set_index("View")

```

display(df\_calib.style.set\_caption("Diagnostik Kalibrasi — Test Set (Ekspansi)"))

*# Interpretasi*

```

print("\nInterpretasi:")
for view, ece, bs, tl in [("FRONT", ece_front, bs_front, hist_front["train_loss"]),
                          ("SIDE", ece_side, bs_side, hist_side["train_loss"])[-1]]:
    if ece > 0.10 and tl < 0.10:
        diag = "Overconfidence kemungkinan OVERFITTING"

```

```
elif ece < 0.05:
```

```
    diag = "Well-calibrated — kemungkinan LEGITIMATE overconfid
```

```
else:
```

```
    diag = "Ambiguous — perlu ablasi regularisasi lebih lanjut"
```

```
    print(f" [{view}] train_loss = {tl:.5f}, ECE = {ece:.5f} → {diag}")
```

```
        ECE    Brier Score    ECE Status    BS Status
```

```
View
```

```
Front 0.064680 0.080810    △□ SEDANG ✓ BAIK
```

```
Side 0.057250 0.108750    △□ SEDANG △□ SEDANG
```

```
    Diagnostik Kalibrasi — Test Set (Eksperimen 6)
```

Interpretasi:

```
[FRONT] train_loss = 0.14960, ECE = 0.06468 → Ambiguous — perlu ab
```

```
[SIDE] train_loss = 0.12870, ECE = 0.05725 → Ambiguous — perlu abla
```

In [15]:

```
# — Reliability Diagram (Calibration Curve) —————
```

```
fig, axes = plt.subplots(1, 2, figsize = (12, 5))
```

```
fig.suptitle("Reliability Diagram — Kalibrasi Probabilitas (Eksperimen 6",  
             fontsize = 13, fontweight = "bold")
```

```
reliability_diagram(axes[0], scores_front, labels,
```

```
    f"Front View\nECE = {ece_front:.4f} | Brier = {bs_front:.4f}",
```

```
    COLORS["front"])
```

```
reliability_diagram(axes[1], scores_side, labels,
```

```
    f"Side View\nECE = {ece_side:.4f} | Brier = {bs_side:.4f}",
```

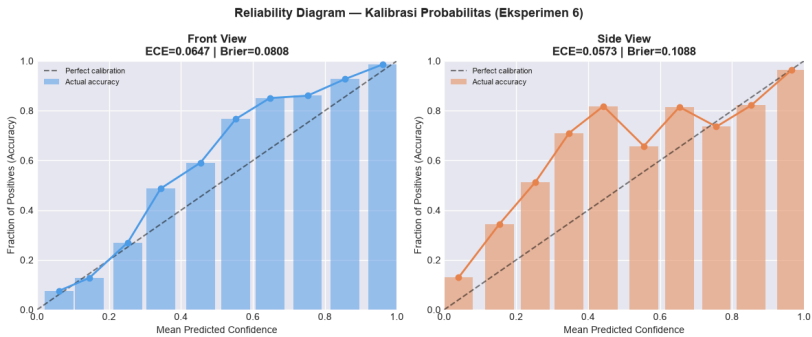
```
    COLORS["side"])
```

```
plt.tight_layout()
```

```
plt.savefig(RESULTS_DIR / "exp6_reliability_diagram.png", dpi = 150, bb
```

```
plt.show()
```

```
print(" ✓ Disimpan ke results/exp6_reliability_diagram.png")
```



✓ Disimpan ke results/exp6\_reliability\_diagram.png

## 9. Ringkasan Eksperimen

In [ ]:

# — *Cetak ringkasan eksperimen* —

```
print(" " * 60)
```

```
print("RINGKASAN EKSPERIMEN 6 (Config v5)")
```

```
print(" " * 60)
```

```
print(f"\n🔪 Model: EfficientNetV2-S (Binary Classification)")
```

```
print(f"  NUM_STAGES_TO_FREEZE : 5 (v5)")
```

```
print(f"  WEIGHT_DECAY          : 1e-3 (v5)")
```

```
print(f"  DROPOUT                : 0.6 (v5)")
```

```
print(f"  RandomRotation         : 20° (v5)")
```

```
print(f"  ColorJitter            : 0.4 (v5)")
```

```
print(f"\n📊 SINGLE-VIEW — Test Set:")
```

```
print(f"  Front : Acc = {metrics_front['accuracy']:.4f} | F1 = {metrics_front['f1_macro']:.4f}")
```

```
print(f"  Side  : Acc = {metrics_side['accuracy']:.4f} | F1 = {metrics_side['f1_macro']:.4f}")
```

```
print(f"\n🔗 FUSION — Test Set (paired frames):")
```

```
for key, label in [(f"average_fusion", "Average (50:50)", "adaptive_fusion"),  
                  (f"fusion_{key}_v1", "Fusion v1", "fusion_v1")]:  
    m = fusion[key]
```

```
    print(f"  {label}: Acc = {m['accuracy']:.4f} | F1 = {m['f1_macro']:.4f}")
```

```
best_key = max(fusion, key=lambda k: fusion[k]["f1_macro"])
```

```
print(f"\n🏆 Best Method: {fusion[best_key]['method']} (F1 = {fusion[best_key]['f1_macro']:.4f})")
```

```
print(" " * 60)
```

=====

## RINGKASAN EKSPERIMEN 6 (Config v5)

=====

✂ Model: EfficientNetV2-S (Binary Classification)

NUM\_STAGES\_TO\_FREEZE : 5 (v5)

WEIGHT\_DECAY : 1e-3 (v5)

DROPOUT : 0.6 (v5)

RandomRotation : 20° (v5)

ColorJitter : 0.4 (v5)

📊 SINGLE-VIEW — Test Set:

Front : Acc=0.8979 | F1=0.8375 (epoch 24)

Side : Acc=0.8502 | F1=0.7381 (epoch 23)

✂ FUSION — Test Set (paired frames):

Average (50:50): Acc=0.9062 | F1=0.8287

Adaptive : Acc=0.9062 | F1=0.8287

🏆 Best Method: Single-view FRONT (F1=0.8293)

=====

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more information.

View Jupyter [log](command:jupyter.viewOutput) for further details.

In [ ]: